Assignment 3: Understanding Algorithm Efficiency and Scalability

Rachana Peddi

University of the Cumberlands

MSCS-532-M70: Algorithms and Data Structures

Introduction

This assignment evaluates the efficiency and scalability of Randomized Quicksort and hashing with separate chaining. The work combines implementation, theoretical analysis, and empirical testing. Randomized Quicksort is compared with a deterministic version that always chooses the first element as the pivot. The comparison uses random, already sorted, reverse-sorted, and repeated-element inputs. A separate hash-table implementation demonstrates insertion, search, update, deletion, collision handling through chaining, load-factor monitoring, and dynamic resizing.

The purpose of the experiments is not only to confirm that the algorithms work, but also to connect observed behavior with asymptotic analysis. The timing measurements in this report were produced by the Python programs created for this assignment and executed locally.

Part 1: Randomized Quicksort

Implementation

The Randomized Quicksort implementation selects a pivot uniformly at random from the current subarray. It partitions the values into three groups: values smaller than the pivot, values equal to the pivot, and values greater than the pivot. The two unequal groups are recursively sorted and then combined with the equal group. Keeping equal values together is useful for inputs containing many duplicates because identical values do not need to be recursively partitioned one at a time.

The base case returns a copy of an array containing zero or one element. The implementation was tested with a random array, repeated elements, an already sorted array, a reverse-sorted array, an empty array, and a single-element array. These tests verify both normal behavior and important edge cases.

Figure 1

Randomized Quicksort edge-case execution results.

```
PS C:\Users\Rachana Reddy\Desktop> cd Assignment3_Algorithms
PS C:\Users\Rachana Reddy\Desktop\Assignment3_Algorithms> python quicksort.py
Random array:
  Original: [10, 7, 8, 9, 1, 5]
  Sorted:   [1, 5, 7, 8, 9, 10]

Repeated elements:
  Original: [4, 2, 4, 1, 2, 4, 3]
  Sorted:   [1, 2, 2, 3, 4, 4, 4]

Already sorted:
  Original: [1, 2, 3, 4, 5, 6]
  Sorted:   [1, 2, 3, 4, 5, 6]

Reverse sorted:
  Original: [6, 5, 4, 3, 2, 1]
  Sorted:   [1, 2, 3, 4, 5, 6]

Empty array:
  Original: []
  Sorted:   []

Single element:
  Original: [42]
  Sorted:   [42]

PS C:\Users\Rachana Reddy\Desktop\Assignment3_Algorithms>
```

Average-Case Analysis

For a subarray of size n, partitioning examines the elements relative to the selected pivot, so the partitioning work is linear, $\Theta(n)$. Because the pivot is selected randomly, its rank is equally likely to occur at any position. The expected running time can therefore be expressed by the recurrence $E[T(n)] = (1/n)\ \Sigma(E[T(k)] + E[T(n-k-1)]) + \Theta(n)$, where k ranges from 0 to n-1 (Cormen et al., 2022).

The random choice does not guarantee an exactly balanced split on every recursive call. Instead, it makes persistently poor pivot choices unlikely. In expectation, the recursion tree has logarithmic depth, and each level performs a total of $\Theta(n)$ partitioning work across its subproblems. Multiplying $\Theta(n)$ work per level by an expected $\Theta(\log n)$ number of levels gives an average expected running time of $\Theta(n \log n)$ (Cormen et al., 2022).

The worst case remains $\Theta(n^2)$ if successive pivots repeatedly create extremely unbalanced partitions. The benefit of randomization is that this behavior is no longer tied predictably to a particular input ordering. By contrast, choosing the first element as the pivot makes already sorted and reverse-sorted arrays particularly unfavorable because each partition can contain n - 1 elements on one side and zero on the other (Cormen et al., 2022).

Empirical Comparison

The empirical test compared Randomized Quicksort with Deterministic Quicksort, where the deterministic implementation always uses the first element as the pivot. Four input distributions were tested at sizes 100, 500, 1,000, and 2,000. Each reported time is the average of five trials. The test also verified each result against Python's sorted result before accepting the timing measurement.

Table 1

Average execution time over five trials.

Distribution	n	Randomized (s)	Deterministic (s)
Random	100	0.000247	0.000146
Random	500	0.000830	0.000563
Random	1000	0.002049	0.001182
Random	2000	0.003229	0.003273
Sorted	100	0.000241	0.000472
Sorted	500	0.000745	0.009679
Sorted	1000	0.001188	0.033202
Sorted	2000	0.002821	0.158595
Reverse Sorted	100	0.000191	0.000450
Reverse Sorted	500	0.001656	0.028127
Reverse Sorted	1000	0.020438	0.064000
Reverse Sorted	2000	0.003803	0.223421
Repeated Elements	100	0.000075	0.000035
Repeated Elements	500	0.000187	0.000141
Repeated Elements	1000	0.000264	0.000237
Repeated Elements	2000	0.000574	0.000430

For random input, the two implementations were relatively close. At n = 2,000, Randomized Quicksort required 0.003229 seconds and Deterministic Quicksort required 0.003273 seconds. At smaller random sizes, the deterministic version was sometimes slightly

faster. This does not contradict the theoretical analysis: randomization adds pivot-selection overhead, and a random input can already provide reasonable partitions for a first-element pivot.

The sorted and reverse-sorted inputs show the strongest difference. At n = 2,000, the deterministic implementation required 0.158595 seconds on sorted input compared with 0.002821 seconds for the randomized implementation. On reverse-sorted input, deterministic Quicksort required 0.223421 seconds compared with 0.003803 seconds for Randomized Quicksort. These results are consistent with the $\Theta(n^2)$ behavior expected when a first-element pivot repeatedly produces highly unbalanced partitions.

Repeated-element inputs were fast for both implementations. At n = 2,000, Randomized Quicksort took 0.000574 seconds and Deterministic Quicksort took 0.000430 seconds. The three-way partitioning used in both implementations places values equal to the pivot into a separate group, so large groups of duplicate values do not generate the same deep recursion that would occur in a simple two-way partition implementation.

One individual measurement, Randomized Quicksort on reverse-sorted input at n = 1,000, was noticeably higher than the neighboring randomized measurements. Timing experiments can be affected by random pivot choices and normal runtime/system variation. Because each value was averaged over five trials, the table is more informative than a single run, but repeating the complete experiment more times would provide a more stable statistical estimate.

Figure 2

Terminal output from the empirical Quicksort comparison.

```
PS C:\Users\Rachana Reddy\Desktop\Assignment3_Algorithms> python performance_test.py
Randomized vs. Deterministic Quicksort
Average execution time over 5 trials

Distribution           Size    Randomized (s)     Deterministic (s)
-----------------------------------------------------------------
Random                  100         0.000247             0.000146
Random                  500         0.000830             0.000563
Random                 1000         0.002049             0.001182
Random                 2000         0.003229             0.003273

Sorted                  100         0.000241             0.000472
Sorted                  500         0.000745             0.009679
Sorted                 1000         0.001188             0.033202
Sorted                 2000         0.002821             0.158595

Reverse Sorted          100         0.000191             0.000450
Reverse Sorted          500         0.001656             0.028127
Reverse Sorted         1000         0.020438             0.064000
Reverse Sorted         2000         0.003803             0.223421

Repeated Elements       100         0.000075             0.000035
Repeated Elements       500         0.000187             0.000141
Repeated Elements      1000         0.000264             0.000237
Repeated Elements      2000         0.000574             0.000430
```

Part 2: Hashing with Chaining

Implementation

The hash table uses separate chaining for collision resolution. The table is represented as a list of buckets, and each bucket is a list of key-value pairs. A deterministic polynomial string hash is implemented directly in the program. Starting with a hash value of zero, each character is incorporated using base 31, and modulo the current table capacity keeps the result within the valid bucket range. If multiple keys map to the same index, their entries remain in the same bucket chain rather than overwriting one another.

The insert operation searches the selected bucket for an existing key. If the key already exists, its value is updated otherwise, a new key-value pair is appended and the element count

increases. Search scans only the selected chain and returns the associated value when the key is found. Delete similarly scans the selected chain, removes the matching pair, and decreases the element count. Missing searches return None, while deletion reports False when a key is absent.

The implementation also tracks the load factor $\alpha = n/m$, where n is the number of stored entries and m is the number of buckets. When the load factor becomes greater than 0.75, the table doubles its capacity and rehashes the existing entries. Rehashing is necessary because the bucket index depends on the table capacity.

Figure 3

Final hash-table insert, resize, search, update, and chaining results.

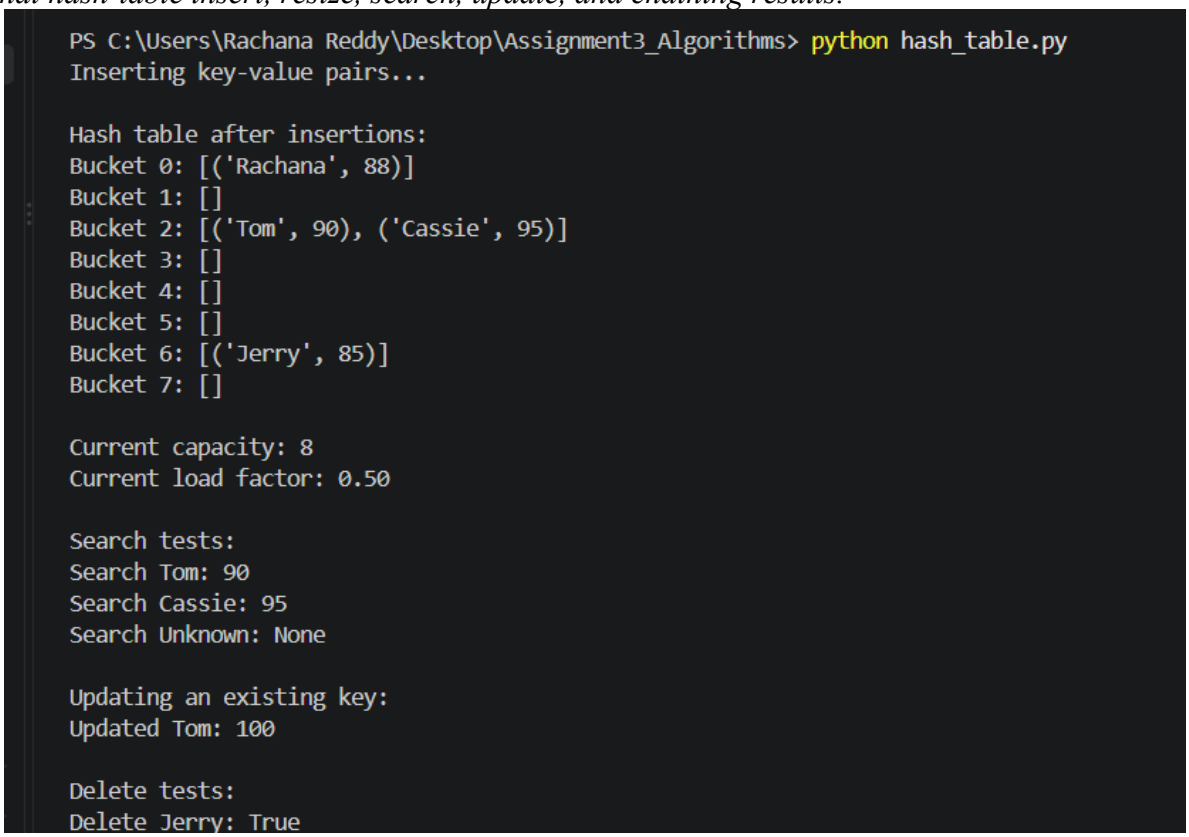


Figure 4

Final hash-table deletion results and chained bucket contents.



Hash Table Analysis

Under the assumption of simple uniform hashing, keys are distributed approximately evenly among the m buckets. The expected chain length is the load factor $\alpha = n/m$. Computing the bucket index is constant-time work, after which an operation examines the selected chain. Therefore, expected search and delete time is $\Theta(1 + \alpha)$. Insertion is also expected $\Theta(1 + \alpha)$ in this implementation because the selected chain is checked first to determine whether the key already exists (Cormen et al., 2022).

If α remains bounded by a constant, the expected cost of these operations is effectively constant, $\Theta(1)$. If the table is allowed to become too full, chains become longer and more comparisons are required. In the extreme case where many keys collide into one bucket, an

operation can degrade toward Θ(n). This is why both the quality of the hash distribution and control of the load factor matter (Cormen et al., 2022).

Dynamic resizing helps maintain a low load factor. In this implementation, the initial capacity is four. Inserting Tom, Jerry, Cassie, and Rachana causes the load factor to exceed 0.75, so the capacity doubles to eight and all entries are rehashed. The observed output then reports a capacity of 8 and a load factor of 0.50. The final output also demonstrates chaining because more than one key can occupy the same bucket without data loss.

Resizing itself requires Θ(n) time because existing entries must be reinserted into the larger table. However, resizing is not performed on every insertion. When capacity grows geometrically, such as doubling, the expensive resize cost is spread across many insertions, giving insertion an expected amortized constant-time behavior when the load factor remains controlled (Cormen et al., 2022).

Code Quality and Testing

The project separates the major responsibilities into quicksort.py, hash_table.py, and performance_test.py. Functions and methods have focused responsibilities and short documentation strings. The Quicksort implementation includes explicit edge-case tests, while the hash-table driver exercises insertion, search, update, deletion, a missing key, load-factor calculation, resizing, and chaining. The empirical comparison verifies every sorting result before recording the timing, reducing the chance of treating an incorrect implementation as a performance result.

The implementation uses a custom deterministic polynomial string hash with base 31. This provides a clear, reproducible mapping from string keys to bucket indices and avoids relying on Python's process-randomized built-in hash values for this demonstration. It is

described as a polynomial hash rather than as a universal-hashing family. Separate chaining handles any collisions that remain, while dynamic resizing helps keep the load factor controlled.

Conclusion

The theoretical and empirical results show why algorithm design choices affect scalability. Randomized Quicksort retains a $\Theta(n^2)$ worst case, but random pivot selection makes performance less dependent on the original ordering of the input and gives an expected $\Theta(n \log n)$ running time. The local experiments strongly demonstrated the weakness of a first-element deterministic pivot on sorted and reverse-sorted data, while also showing that randomized selection is not necessarily faster on every small or favorable input.

The hash-table implementation demonstrates how separate chaining preserves entries when collisions occur and how load factor affects expected operation cost. Maintaining a bounded load factor through dynamic resizing keeps expected search, insertion, and deletion efficient. Together, the two parts illustrate the importance of combining asymptotic analysis with empirical measurements when evaluating algorithm efficiency and scalability.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms (4th ed.).* MIT Press.